Dynamic eMode Router: Optimizing Capital Efficiency and Risk Management in Aave V3 (v0.2)

[Rishav]
anubrat23@gmail.com

Version 0.2 — March 5, 2026

Abstract

Aave V3's efficiency mode (eMode) allows users to borrow up to 97% loan-to-value (LTV) against correlated assets, significantly improving capital efficiency compared to standard 80% LTV limits. However, static eMode configurations create a tradeoff: users must choose between high borrowing power for correlated assets or diversification flexibility across asset categories. This paper presents a Dynamic eMode Router, a smart contract system that automatically manages eMode transitions based on portfolio composition and market conditions, eliminating the need for manual category switching while maintaining optimal capital efficiency.

Our router monitors user positions and market volatility to determine the optimal eMode category in real-time. When a user's portfolio becomes concentrated in assets from a single eMode category, the router automatically enables that category to maximize borrowing capacity. Conversely, when diversification increases or market volatility exceeds safe thresholds, the router switches to standard mode to prevent liquidation cascades.

Initial simulation results demonstrate that the Dynamic eMode Router improves capital efficiency by X% while reducing liquidation risk by Y% compared to static eMode configurations. The system maintains these improvements even during high volatility periods, as validated through Monte Carlo simulations and historical stress tests. This work contributes to the growing body of research on adaptive DeFi protocols that balance capital efficiency with risk management.

Keywords: Decentralized Finance, Aave, Capital Efficiency, Risk Management, Smart Contracts, eMode

1 Introduction

The evolution of Decentralized Finance (DeFi) lending protocols has consistently focused on two competing objectives: maximizing capital efficiency and minimizing liquidation risk. Aave V3, one of the largest lending protocols with over $10 billion in total value locked, introduced efficiency mode (eMode) to address this challenge by allowing higher loan-to-value ratios for correlated assets within predefined categories [1].

Under standard Aave operation, users can borrow up to 80% of their collateral value across any supported assets. However, eMode enables borrowing up to 97% LTV when both collateral and borrowed assets belong to the same category (e.g., ETH Correlated assets including stETH, wstETH, and rETH). This 17 percentage point increase in borrowing capacity represents a significant improvement in capital efficiency, particularly valuable for sophisticated DeFi strategies such as looping, yield farming, and arbitrage.

Despite these benefits, eMode adoption remains suboptimal due to its static nature. Users must manually select an eMode category before depositing collateral, creating a commitment that is costly to change. This design forces users into a binary choice: optimize for capital efficiency by concentrating in a single asset category, or maintain portfolio diversification at the

cost of reduced borrowing power. The manual switching process also introduces timing risks, as users may fail to adjust their eMode category when market conditions change or when their portfolio composition evolves.

1.1 Problem Statement

The current eMode implementation suffers from three key limitations:

1. Static Configuration Burden: Users must predict their future borrowing needs and market conditions when selecting an eMode category. This prediction problem is particularly challenging given the volatility of cryptocurrency markets and the evolving landscape of available DeFi strategies.

2. Inefficient Capital Utilization: Users who maintain diversified portfolios cannot access eMode benefits even when a significant portion of their assets would qualify for a specific category. Conversely, users in eMode cannot diversify without losing efficiency benefits, creating unnecessary concentration risk.

3. Manual Risk Management: Current eMode requires users to monitor market conditions and manually adjust their category selection during periods of high volatility or correlation breakdown. This manual process is error-prone and may be too slow during rapid market movements.

1.2 Contribution

This paper presents the Dynamic eMode Router, an autonomous smart contract system that addresses these limitations by:

- **Automated Category Selection:** Continuously analyzing user portfolio composition to determine the optimal eMode category without manual intervention
- **Risk-Adjusted Switching:** Incorporating real-time volatility metrics and correlation analysis to prevent eMode activation during unsafe market conditions
- **Capital Efficiency Optimization:** Maximizing borrowing capacity across diverse portfolio compositions while maintaining liquidation safety margins
- **Composable Architecture:** Providing a standardized interface for other protocols to integrate dynamic eMode functionality

Our evaluation demonstrates that the Dynamic eMode Router improves capital efficiency by X% and reduces liquidation probability by Y% compared to static eMode configurations, while maintaining safety even during extreme market stress scenarios such as the May 2022 LUNA collapse.

The remainder of this paper is structured as follows: Section 2 provides technical background on Aave V3 and eMode mechanics; Section 3 details the router architecture and algorithms; Section 4 analyzes security properties and invariants; Section 5 presents empirical results from Monte Carlo simulations and historical backtests; and Section 6 discusses implications and future work.

2 Background

This section provides technical foundations for understanding the Dynamic eMode Router by reviewing Aave V3's architecture, eMode mechanics, and the metrics that govern capital efficiency in lending protocols.

2.1 Aave V3 Protocol Overview

Aave V3 is a decentralized lending protocol that enables users to deposit crypto assets as collateral to borrow other assets, with interest rates determined algorithmically based on supply and demand. The protocol manages user positions through a core smart contract called the *Pool*, which maintains:

- **Reserves:** Each supported ERC20 token corresponds to a reserve with configuration parameters (decimals, LTV, liquidation thresholds, interest rate models)
- **User Configurations:** Bitmaps tracking which reserves each user has deposited or borrowed from
- **aTokens:** Yield-bearing tokens representing deposits; balances automatically accrue interest via scaled balances and normalized income
- **Debt Tokens:** Variable-rate (scaled) and stable-rate debt tokens representing borrowing obligations
- **Price Oracle:** Real-time price feed for all assets, crucial for collateral valuation and liquidation calculations

A user's *health factor* (HF) is the primary safety metric in Aave. Formally:

$$\text{HF} = \frac{\text{Collateral Value} \times \text{Avg Liquidation Threshold}}{\text{Total Debt}} \tag{1}$$

When HF drops below 1.0, the user enters liquidation, and third-party liquidators can repay debt to seize collateral at a discount. Users must maintain HF > 1.0 to avoid liquidation; prudent users maintain much higher buffers.

2.2 Efficiency Mode (eMode) Mechanics

Aave V3 introduces *efficiency mode (eMode)*, a feature allowing higher leverage for correlated assets. Each eMode category (identified by a numerical ID) groups related assets and defines higher borrowing parameters:

- **Standard Mode (ID = 0):** All users start here. Allows up to 80% LTV universally
- **eMode Categories (ID $\geq$ 1):** Specialized modes for correlated asset groups (e.g., ETH staking derivatives: WETH, stETH, wstETH, rETH). Allow up to 97% LTV when both collateral and borrowed assets belong to the same category

The critical distinction: LTV (Loan-to-Value) determines maximum borrowing capacity. With 97% eMode LTV versus 80% standard LTV, a user can potentially borrow 17 percentage points more of their collateral, significantly improving capital efficiency for concentrated portfolios.

Aave enforces *category membership* at the reserve level: each reserve has an eMode category ID. Users see the benefits of high LTV only if:

1. The user's eMode is set to that category
2. Both borrowed and collateral assets belong to that category

If a user is in ETH eMode but borrows USDC (which has category ID 0), that USDC borrowing uses the standard 80% LTV, limiting the efficiency gains. This creates the inefficiency our router addresses.

2.3 Capital Efficiency Metrics

Capital efficiency in lending is measured by leverage: the ratio of borrowed assets to collateral. We define:

$$\text{LTV} = \frac{\text{Maximum Borrowing Capacity}}{\text{Total Collateral Value}} \tag{2}$$

$$\text{Available Borrows} = \text{Collateral} \times \text{LTV} - \text{Current Debt} \tag{3}$$

For diversified users unable to use eMode, available borrowing is limited to:

$$\text{Available Borrows}_{\text{standard}} = \sum_i (\text{Collateral}_i) \times 0.80 - \text{Total Debt} \tag{4}$$

For concentrated users in eMode, borrowing increases to:

$$\text{Available Borrows}_{\text{eMode}} = \sum_i (\text{Collateral}_i) \times 0.97 - \text{Total Debt} \tag{5}$$

The difference, $(0.97 - 0.80) \times \text{Total Collateral} = 0.17 \times \text{Total Collateral}$, represents the *unlocked capital efficiency* from eMode switching. However, users can only capture this if they (1) manually discover the optimal category, and (2) commit to holding only correlated assets.

The Dynamic eMode Router automates this discovery and switching process while maintaining safety through health factor buffers and conservative switching logic, as detailed in Section 3.

3 System Design

This section describes the architecture, decision algorithm, and safety mechanisms of the Dynamic eMode Router. Our implementation closely mirrors Aave V3's internal 'GenericLogic' library to ensure correctness and consistency with Aave's own health factor calculations.

3.1 Router Architecture

The DynamicEModeRouter is deployed as a single smart contract that interfaces with Aave V3's Pool contract. It exposes three categories of functions:

3.1.1 Data Inspection Functions

- **getUserData(address user)** — Returns a snapshot of the user's complete position: total collateral (base currency), total debt, available borrows, current liquidation threshold, current LTV, health factor, and current eMode category
- **getUserReservePositions(address user)** — Lists every reserve the user interacts with, including asset address, whether it is collateral/borrowing, LTV, liquidation threshold, eMode category membership, and current price

These functions are `view`-only and incur no gas cost; they allow external systems (keepers, frontends, other contracts) to inspect state before triggering optimization.

3.1.2 Simulation Engine

- **simulateEMode(address user, uint8 targetEModeId)** — Performs a *dry-run* calculation: given a hypothetical eMode category, this function computes what the user's health factor, liquidation threshold, and available borrows would be. It does **not** modify any state; it purely calculates and returns the simulation result.

The simulation engine is the technical core of our system. It replicates Aave V3's 'calculateUserAccountData()' function (from GenericLogic library) but with a forced eMode override, allowing us to predict HF changes before execution.

3.1.3 Optimization Action

- **optimizeEMode()** — The main entry point. This function scans all eMode categories (0 through 10), simulating each, and identifies the category that maximizes available borrows while staying safe (HF > 1.05). If a different category improves borrowing, it calls Aave's 'pool.setUserEMode()' to execute the switch. Emits an 'EModeOptimized' event for transparency.

3.1.4 Contract Invariants

- **ReentrancyGuard protection:** The 'optimizeEMode()' function is protected by ReentrancyGuard to prevent flash loan attacks or other reentrancy vectors
- **Only msg.sender is optimized:** The optimization always operates on 'msg.sender', ensuring users cannot optimize others' positions
- **No custody of funds:** The router never holds user assets; it only reads balances and configures eMode settings

3.2 Decision Algorithm

Algorithm **??** formalizes the optimization logic.

Key decision criteria:

1. **Safety first:** Only switch to a category if the simulated HF exceeds HF_BUFFER (1.05 WAD), providing a 5% safety margin
2. **Borrowing improvement:** Only switch if available borrows actually increase, preventing no-op or harmful switches
3. **Greedy best:** Among all safe categories, pick the one with the highest available borrows (capital efficiency maximization)
4. **Idempotent:** If the best category is the current category, skip the transaction (no unnecessary on-chain operations)

3.2.1 Simulation Calculation

The simulateEMode function computes health factor for a target eMode category by iterating over all reserves the user interacts with:

1. For each collateral asset: multiply balance by (eMode LTV if asset matches target category, else standard LTV)
2. Sum weighted LTV and weighted liquidation threshold

Algorithm 1 OptimizeEMode Decision Algorithm

```
function OptimizeEMode(user)
    currentEMode ← pool.getUserEMode(user)
    bestEModeId ← currentEMode
    bestAvailableBorrows ← simulateEMode(user, currentEMode).availableBorrows
    for categoryId := 0 to MAX_EMODE_CATEGORY do
        if categoryId == currentEMode then
            continue
        end if
        sim ← simulateEMode(user, categoryId)
        if sim.healthFactor > HF_BUFFER and sim.availableBorrows > bestAvailableBorrows then
            bestAvailableBorrows ← sim.availableBorrows
            bestEModeId ← categoryId
        end if
    end for
    if bestEModeId ≠ currentEMode then
        pool.setUserEMode(bestEModeId)
        emit EModeOptimized(user, currentEMode, bestEModeId)
    end if
end function
```

3. For each borrowing asset: accumulate total debt (scaled variable debt + plain stable debt)
4. Calculate: avgLtv = weightedLtv / totalCollateral, avgLiqThreshold = weightedLiqThreshold / totalCollateral
5. Health Factor: $\text{HF} = \frac{\text{totalCollateral} \times \text{avgLiqThreshold}}{\text{totalDebt}}$
6. Available Borrows: $\max(0, \text{totalCollateral} \times \text{avgLtv} - \text{totalDebt})$

This mirrors Aave's GenericLogic.calculateUserAccountData() precisely, ensuring our simulations match what Aave will enforce on-chain.

3.3 Risk Management Framework

The router incorporates multiple layers of safety to prevent user harm:

3.3.1 Health Factor Buffer

$$\text{HF}_{\text{required}} = 1.05 \text{ WAD} \tag{6}$$

By requiring HF > 1.05 before switching, we provide a 5% safety margin. This buffer prevents oscillations near the liquidation boundary and allows for small price movements without immediate liquidation risk. Empirically, 1.05 is conservative enough to survive typical intraday volatility while still unlocking meaningful capital efficiency gains.

3.3.2 Borrow Improvement Check

The router only executes a switch if new available borrows are strictly greater than current available borrows:

$$\text{AvailableBorrows}_{\text{new}} > \text{AvailableBorrows}_{\text{current}} \tag{7}$$

This prevents wasteful switching to categories that offer no benefit and ensures every switch improves the user's position.

3.3.3 Pre-Execution Simulation vs. Post-Execution Reality

Because on-chain conditions (prices, interest rates, user balances) can change between simulation and execution, the router uses a conservative approach:

- Simulations use live Aave state (reserves, prices, user balances) at the time of the transaction
- The HF buffer (1.05) absorbs small changes in price or accrued interest
- If a user's position becomes unhealthy between simulation and execution (e.g., massive liquidation event), Aave's 'setUserEMode()' may still fail, but it will revert the entire transaction atomically

3.3.4 Reentrancy Protection

The optimizeEMode function uses OpenZeppelin's ReentrancyGuard. During or after calling pool.setUserEMode(), if a malicious callback attempts to re-enter optimizeEMode, the guard blocks the second call. This prevents flash loan attacks where an attacker could:

1. Borrow a large amount via flash loan
2. Trigger router optimization
3. Have the router call pool.setUserEMode(), triggering a callback
4. In the callback, re-enter the router and manipulate state

ReentrancyGuard prevents step (4) by reverting the second call.

3.3.5 No Manual Price Manipulation

The router reads asset prices directly from Aave's IPriceOracleGetter interface. We do not accept user-supplied prices or make assumptions about price bounds. This ensures we inherit Aave's oracle security model and are immune to price oracle attacks.

3.3.6 Reserve Whitelist via eMode Membership

We do not hardcode which assets are safe. Instead, we only consider assets that Aave itself has marked as members of specific eMode categories. If Aave governance adds a new asset or removes one from a category, the router automatically respects that change. This delegates reserve safety decisions to Aave's governance.

Summary: The risk management framework combines conservative health factor buffers, pre-execution checks, atomic transactions, and delegation to Aave's audited oracle and governance processes. Together, these mechanisms ensure the router enhances capital efficiency without introducing new risk vectors.

4 Security Analysis

4.1 System Invariants

4.2 Attack Vectors and Mitigations

4.3 Formal Verification

5 Evaluation

5.1 Experimental Setup

5.2 Capital Efficiency Results

5.3 Risk Analysis

5.4 Stress Testing

5.5 Gas Cost Analysis

6 Conclusion

6.1 Summary of Contributions

6.2 Limitations

6.3 Future Work

References

[1] Aave Companies. Aave protocol v3. https://github.com/aave/aave-v3-core/blob/master/techpaper/Aave_V3_Technical_Paper.pdf, 2022. Accessed: March 2026.